

React and TypeScript Engineering Guide

Maintainable product interfaces for Cocoarynth Trace

Document ID	ENG-WEB-01
Status	Approved
Effective date	2026-09-15
Owners	Trace Web Platform and Developer Experience
Applies to	React and TypeScript production code

1. Engineering intent

Cocoarynth Trace interfaces carry sourcing, production, certification, and shipment facts. Code should make the ownership and movement of those facts easy to inspect. Prefer explicit data boundaries, small components, predictable state, and tests that describe user-visible behavior.

Required: New frontend production code uses TypeScript in strict mode, functional React components, semantic HTML, and the repository's configured formatter and linter. Do not weaken a compiler or lint rule only to make a change pass.

2. TypeScript

- Model domain concepts with named types. Use unions for finite states such as "draft" | "issued" | "superseded".
- Prefer unknown at untrusted boundaries and narrow it through validation. Avoid any; an exception requires a focused explanation in review.
- Keep nullability honest. Do not substitute empty strings or zeroes when a value is genuinely absent.
- Use immutable inputs and return values by default. A function that mutates shared state must make that behavior obvious in its name and ownership.
- Infer local types when the result is clear. Annotate exported functions, domain models, state shapes, and boundaries where inference would hide a contract.
- Do not duplicate server schemas by hand when the project provides generated or shared contract types.

```
type PassportStatus = "draft" | "issued" | "superseded";

type OriginPassportSummary = {
  id: string;
  batchNumber: string;
  status: PassportStatus;
  issuedAt: string | null;
};
```

3. Components and module boundaries

- Give each component one recognizable UI responsibility. Extract a component when it owns behavior, repeated structure, or a testable concept, not merely to shorten a file.
- Keep route and feature modules responsible for data loading and workflow state. Keep reusable presentational components independent of route-specific APIs.
- Use composition and ordinary props before introducing context. Context is appropriate for stable cross-cutting dependencies, not as a general state store.
- Do not create wrapper components that only rename native HTML without adding semantics, policy, or repeated behavior.
- Preserve repository naming and folder conventions. Component names use PascalCase; hooks begin with use; event handlers describe the event or outcome.

4. State, effects, and data flow

- Keep state as close as possible to the components that coordinate it. Lift it only when two owners need one source of truth.
- Derive values during rendering when they can be calculated from props or state. Do not mirror derived values into another state variable.
- Use effects to synchronize with systems outside React, such as network subscriptions, browser APIs, or third-party widgets. Do not use effects as a substitute for event handling or derivation.
- Abort or ignore obsolete asynchronous work so an older request cannot overwrite newer state.
- Represent loading, empty, error, success, and permission-denied states deliberately. A blank panel is not an error strategy.
- Keep server state in the project's data-fetching layer when one exists; do not build a second cache inside component state.

5. Forms and user actions

- Associate every input with a visible label and expose validation errors programmatically.
- Validate at the boundary and preserve the user's entered values after a recoverable error.
- Disable or guard repeated destructive submissions. Show progress without replacing a clear button label with unexplained animation.
- Use native buttons for actions and links for navigation. Keyboard operation must follow the same workflow as pointer operation.

- Confirm high-impact destructive actions and describe the object affected. Do not use confirmation dialogs for routine reversible actions.

6. Accessibility in component code

- Start with native semantic elements. Add ARIA only when native semantics cannot express the interaction.
- Preserve a logical heading hierarchy, accessible names, visible keyboard focus, and meaningful document order.
- Do not communicate status through color alone. Announce asynchronous errors and confirmations through an appropriate live region.
- Images that convey information require useful alternative text. Decorative images use an empty alternative.
- Custom widgets require keyboard, focus, and screen-reader behavior equivalent to their native counterpart.

7. Testing expectations

- Unit-test pure transformation and validation logic, especially format conversion and permission decisions.
- Use React Testing Library for behavior at the component boundary. Query by role and accessible name when possible.
- Use Playwright for critical workflows, browser integration, responsive behavior, and accessibility scans.
- Test loading, empty, failure, and recovery paths, not only the successful response.
- Treat snapshots as narrow regression aids. Do not replace assertions about behavior with large unexplained snapshots.
- Automated accessibility checks supplement keyboard, screen-reader, zoom, and human review; they do not prove conformance.

8. Review and delivery

- Keep changes small enough that a reviewer can understand intent, behavior, and risk.
- Include tests with behavior changes and update user-facing or architectural documentation when contracts change.
- Review for correctness, accessibility, security, operability, and consistency before discussing style preferences.
- Release through the normal automated path. A change is not complete until failures are observable and rollback or mitigation is understood.
- After an incident, document the timeline, contributing conditions, what worked, and concrete preventive actions. Focus on systems and learning rather than blame.

9. Patterns to avoid

- Class components in new code without a documented compatibility reason.
- One global component state object for unrelated workflows.
- Effects that update state solely because another state value changed.
- Deep prop drilling solved by immediately adding a global store.
- Visual-only controls built from `div` or `span`.
- Tests coupled to implementation details such as private state or exact component trees.

10. Pull request checklist

1. Types describe the domain and reject invalid states at boundaries.
2. Data ownership and component responsibilities are clear.
3. Loading, empty, error, and success states are usable.
4. Keyboard and accessible-name behavior has been checked.
5. Focused tests cover the changed behavior and failure paths.
6. Operational impact, documentation, and rollout are addressed.